AULA 04
ESTRUTURAS DE CONTROLE

Programação em Python • Parte 2 de 2
Curso de Capacitação em Desenvolvimento Full Stack
Tema: Estruturas de Controle • 8 horas
Professor: Msc. Hygo Sousa de Oliveira

Roteiro da aula

O que vamos ver nesta aula

1. Revisão: o que o código imprime?
2. Exceções: o que são e por que tratar
3. try, except, else e finally
4. Levantando exceções com raise
5. Exemplos guiados para fazer em sala

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Explicar o que é uma exceção**
  Diferença entre erro de sintaxe e erro em execução
- **Tratar falhas com try e except**
  Vários except, else e finally
- **Levantar exceções com raise**
  Validar dados e interromper o fluxo
- **Aplicar o módulo em programas interativos**
  Leitura segura de dados e laços de tentativas

Revisão: o que o código imprime?

Preveja a saída antes de executar

QUESTÃO 1

```
for i in range(3):
    if i == 1:
        continue
    print(i)
```

QUESTÃO 2

```
n = 3
while n > 0:
    n -= 1
    if n == 1:
        break
print(n)
```

EM SALA Peça que a turma escreva a previsão no caderno, compare com os colegas e só então execute.

O que são exceções?

Erros que acontecem durante a execução e interrompem o programa

excecao.py

```
idade = int("vinte")
print(idade)
```

SAÍDA

```
Traceback (most recent call last):
  File "excecao.py", line 1, in <module>
    idade = int("vinte")
ValueError: invalid literal for int() with base 10: 'vinte'
```

NOTA Sem tratamento, a exceção encerra o programa. **Tratar** é capturar a falha e decidir o que fazer.

try / except

Capturando falhas específicas

try_except.py

```
try:
    numero = int(input("Digite um número: "))
    resultado = 10 / numero
    print(f"Resultado: {resultado}")
except ValueError:
    print("Erro: digite um número válido.")
except ZeroDivisionError:
    print("Erro: não é possível dividir por zero.")
```

SAÍDA

```
Digite um número: 0
Erro: não é possível dividir por zero.
```

EM SALA Teste três entradas: `abc`, `0` e `4`. Cada uma segue um caminho diferente.

Principais exceções

Conhecer o nome ajuda a capturar o erro certo

Exceção	Quando ocorre	Exemplo
`ValueError`	Valor inadequado para o tipo	`int("abc")`
`TypeError`	Operação com tipos incompatíveis	`"2" + 3`
`ZeroDivisionError`	Divisão por zero	`10 / 0`
`KeyError`	Chave inexistente em dicionário	`dados["idade"]`
`IndexError`	Índice fora da lista	`lista[10]`
`FileNotFoundError`	Arquivo inexistente	`open("x.txt")`

ATENÇÃO Evite `except:` vazio: ele esconde erros. Capture exceções específicas.

else e finally

O fluxo completo do tratamento de exceções

try
código que pode falhar

→

except
executa se houver erro

→

else
executa se não houver erro

→

finally
executa sempre

completo.py

```
try:
    numero = int(input("Número: "))
    resultado = 10 / numero
except ZeroDivisionError as e:
    print(f"Erro: {e}")
else:
    print(f"Resultado: {resultado}")
finally:
    print("Encerrando operação.")
```

SAÍDA

```
Número: 4
Resultado: 2.5
Encerrando operação.
```

raise: levantando exceções

Você também pode sinalizar erros

raise.py

```
idade = -5
try:
    if idade < 0:
        raise ValueError("Idade não pode ser negativa")
    print("Idade válida")
except ValueError as erro:
    print(f"Erro: {erro}")
```

SAÍDA

```
Erro: Idade não pode ser negativa
```

NOTA `raise` interrompe o fluxo na hora e entrega uma exceção para quem estiver tratando.

Exemplo guiado 1: leitura segura de nota

Laço + try/except + continue + break

DESAFIO Peça uma nota de 0 a 10 e repita até receber um valor numérico dentro da faixa.

nota_segura.py

```
while True:
    texto = input("Nota (0 a 10): ")
    try:
        nota = float(texto)
    except ValueError:
        print("Só números.")
        continue
    if 0 <= nota <= 10:
        break
    print("Fora da faixa.")
print(f"Nota: {nota}")
```

SAÍDA

```
Nota (0 a 10): dez
Só números.
Nota (0 a 10): 11
Fora da faixa.
Nota (0 a 10): 8.5
Nota: 8.5
```

Exemplo guiado 2: sorteio com random

Módulo random e laço de tentativas

DESAFIO Role um dado até sair o 6, mostrando cada tentativa.

dado.py

```
import random
random.seed(7)
tentativas = 0
dado = 0
while dado != 6:
    dado = random.randint(1, 6)
    tentativas += 1
    print(f"Tentativa {tentativas}: {dado}")
print("Tirou 6!")
```

SAÍDA

```
Tentativa 1: 3
Tentativa 2: 2
Tentativa 3: 4
Tentativa 4: 6
Tirou 6!
```

DICA `random.seed(7)` fixa a sequência para a aula ser reproduzível. Sem ele, o resultado muda a cada execução.

Atividades da Aula 4

Módulo 2: exceções e projetos de fixação

- **Ex. 09 – Calculadora com try/except**
  Tratamento de entradas inválidas e divisão por zero
- **Ex. 10 – Jogo de adivinhação**
  Laço `while`, módulo `random` e busca binária
- **Ex. 11 a 14 – Exercícios do módulo**
  Incluem número primo e senha forte
- **Ex. 15 – Desafio final**
  Menu de sistema interativo integrando todos os conceitos do módulo

GIT

Enunciados, instruções de execução e de entrega:
github.com/Hygo/python-fullstack-iteam-2sem

Obrigado(a)!

Próxima aula: Estruturas de dados (listas, tuplas, dicionários e conjuntos)
ITEAM – Instituto Tecnológico Educacional da Amazônia